BTS SIO – OPTION SLAM

Site Test Marchand

Documentation Technique – Épreuve E6

Application web de commerce en ligne (marketplace) développée avec Laravel 12, intégrant un système de paiement Stripe, la gestion d'abonnements d'entreprises et un flux de retrait en magasin sécurisé.

Projet	Site Test Marchand (Marketplace)
Framework	Laravel 12 – PHP 8.2+
Base de données	MariaDB (développement / démo)
Paiement	Stripe Checkout + Stripe Connect
Date du document	Avril 2026
Option BTS	SIO – SLAM

Sommaire

1. Contexte et objectifs du projet
2. Architecture technique
3. Modèle de données (MCD / MLD)
4. Description des modules applicatifs
5. Sécurité
6. Intégration Stripe
7. Tableau des routes
8. Guide d'installation
9. Pistes d'amélioration
10. Glossaire

1. Contexte et objectifs du projet

1.1 Présentation générale

Ce projet a été réalisé dans le cadre du BTS Services Informatiques aux Organisations, option Solutions Logicielles et Applications Métiers (SLAM). Il constitue le site web « test marchand » présenté lors de l'épreuve E6 (Parcours de professionnalisation).

L'application simule une **marketplace multi-vendeurs** permettant à des entreprises inscrites de publier des annonces (listings) et à des clients de passer commande en ligne avec paiement sécurisé via Stripe.

1.2 Objectifs fonctionnels

- Permettre l'inscription et l'authentification des utilisateurs (client et entreprise)
- Offrir un catalogue public d'annonces filtrables par ville et catégorie
- Gérer un panier persistant et un tunnel de paiement via Stripe Checkout
- Permettre aux entreprises de gérer leurs annonces (CRUD) après souscription à un abonnement
- Implémenter un système de retrait en magasin sécurisé par code OTP (pickup code)
- Exposer un espace administrateur pour valider les demandes de passage en compte entreprise
- Vérifier l'identité légale des entreprises via l'API SIRENE (INSEE)

1.3 Utilisateurs cibles (Personas)

Rôle	Description	Actions principales
Client	Acheteur grand public	Parcourir le catalogue, ajouter au panier, payer, suivre ses commandes
Entreprise	Vendeur inscrit et abonné	Publier/gérer des annonces, préparer les commandes, valider les retraits

2. Architecture technique

2.1 Stack technologique

Couche	Technologie	Version	Rôle
Backend	PHP / Laravel	8.2+ / 12.x	Framework MVC, ORM Eloquent, routing, queues
Frontend	Bootstrap / Vite	5.3.8 / 6.x	Composants UI, bundling assets
Base de données	MariaDB	3.x	Stockage données (dev/démo)
Paiement	Stripe PHP SDK	19.2	Checkout, Connect, Webhooks, Subscriptions
Tests	PestPHP	4.3	Tests unitaires et fonctionnels
Qualité code	PHPStan + Pint	2.x / 1.x	Analyse statique, formatage PSR-12
API externe	INSEE SIRENE API	V3.11	Vérification SIRET/SIREN entreprises

2.2 Pattern MVC (Laravel)

L'application suit strictement le pattern **Model–View–Controller** de Laravel :

```
┌──────────────────────────────────────────────┐
│ REQUÊTE HTTP                                 │
└──────────────────────────────────────────────┘
        │
        ▼
┌──────────────────┐
│ routes/web.php   │ ← Routeur Laravel
└──────────────────┘
        │
        ▼
┌──────────────────────┐
│ Middleware Stack     │ ← Auth, Verified, Role Guards
└──────────────────────┘
            │
            ▼
┌──────────────────────┐
│ Controller           │ ← Logique métier
│ (Http/Controllers/…) │
└──────────────────────┘
     │          │
     │          │
     ▼          ▼
┌─────────┐  ┌──────────┐
│ Model   │  │ Service  │ ← PickupCodeService, SireneApiService
│ Eloquent│  └──────────┘
└─────────┘
     │
     ▼
┌─────────┐
│ BDD     │ ← MariaDB / MySQL
└─────────┘
     │
     ▼
┌──────────┐
│ View     │ ← Blade Templates + Bootstrap
└──────────┘
```

2.3 Structure des dossiers

Dossier	Contenu
`app/Http/Controllers/`	Contrôleurs organisés par namespace (Admin, Auth, Company, Front)
`app/Models/`	14 modèles Eloquent (Cart, CartItem, Category, ClientProfile, Commune, CompanyProfile, ConnectionLog, Listing, ListingImage, Order, OrderItem, PickupToken, UpgradeRequest, User)
`app/Services/`	Services métier (PickupCodeService, SireneApiService)
`app/Http/Middleware/`	Middlewares de contrôle d'accès par rôle
`database/migrations/`	27 migrations décrivant l'évolution du schéma BDD
`resources/views/`	Templates Blade (composants, layouts, pages)
`routes/web.php`	Définition de toutes les routes web
`routes/auth.php`	Routes d'authentification (Laravel Breeze)

3. Modèle de données

3.1 Entités principales

Table	Description	Clé primaire	Relations clés
`users`	Tous les utilisateurs (client, entreprise, admin)	id	→ listings, orders, clientProfile, companyProfile
`client_profiles`	Profil détaillé client (adresse, téléphone)	id	→ users (1-1)
`company_profiles`	Profil entreprise (SIRET, Stripe Connect, abonnement)	id	→ users (1-1)
`communes`	Villes et codes postaux	id	-
`listings`	Annonces publiées par les entreprises	id	→ users, categories, listingImages
`listing_images`	Images associées aux annonces	id	→ listings
`categories`	Catégories d'annonces	id	→ listings
`carts`	Paniers d'achat (persistants)	id	→ users, cartItems
`cart_items`	Lignes de panier	id	→ carts, listings
`orders`	Commandes passées (statut, mode de livraison)	id	→ users, orderItems, pickupTokens
`order_items`	Lignes de commande avec statut vendeur	id	→ orders, listings
`pickup_tokens`	Codes OTP sécurisés pour retrait en magasin	id	→ orders

Table	Description	Clé primaire	Relations clés
`upgrade_requests`	Demandes de passage en compte entreprise	id	→ users
`connection_logs`	Historique des connexions et déconnexions (audit sécurité)	id	→ users

3.2 Diagramme relationnel simplifié

```
users ──────┬── client_profiles (1-1)
            ├── company_profiles (1-1) ← SIRET, Stripe Connect
            ├── listings ─── listing_images
            │   └── categories
            ├── carts ─── cart_items ──→ listings
            ├── orders ─── order_items ──→ listings
            │   └── pickup_tokens
            └── upgrade_requests
```

3.3 Champs notables de la table `users`

Champ	Type	Description
role	string	Rôle utilisateur : `client`, `entreprise`, `admin`
is_subscribed	boolean	Abonnement actif (ancienne colonne, migrée vers company_profiles)
subscription_ends_at	timestamp	Date d'expiration de l'abonnement
email_verified_at	timestamp	Vérification d'adresse email

3.4 Champs notables de `company_profiles`

Champ	Type	Description
siret	string(14)	Numéro SIRET de l'entreprise (vérifié via INSEE)
stripe_connect_account_id	string	ID compte Stripe Connect du vendeur
stripe_connect_charges_enabled	boolean	Paiements activés sur le compte Connect
fulfillment_mode	string	Mode de livraison : `pickup` ou `delivery`
ownership_documents	json	Documents justificatifs (kbis, etc.)

3.5 Statuts des commandes

```
pending ──→ paid ──→ ready_for_pickup ──→ picked_up
            └──→ (cancelled)
```

Statut	Déclencheur
`pending`	Commande créée, paiement non encore confirmé
`paid`	Webhook Stripe `checkout.session.completed` reçu
`ready_for_pickup`	L'entreprise marque toutes les lignes comme préparées
`picked_up`	Code OTP validé par l'entreprise lors du retrait

4. Description des modules applicatifs

4.1 Authentification (Laravel Breeze)

L'authentification est gérée par **Laravel Breeze** avec vérification d'e-mail obligatoire (`MustVerifyEmail`). Les routes sont définies dans `routes/auth.php`.

- Inscription, connexion, déconnexion
- Réinitialisation de mot de passe par e-mail
- Vérification d'adresse e-mail avant accès aux fonctionnalités protégées

4.2 Catalogue public (Front/ListingController)

Le catalogue est accessible sans authentification. Il affiche les annonces actives avec filtrage.

- Page d'accueil : annonces en vedette
- `/boutique` : catalogue complet avec recherche et filtres (catégorie, ville)
- Autocomplétion des villes via `/boutique/villes/search`
- Fiche détail : `/listings/{id}`

4.3 Panier (Front/CartController)

Le panier est persistant en base de données, associé à l'utilisateur ou à la session :

```
GET  /cart        → Afficher le panier
POST /cart/add/{listing}  → Ajouter un article
PATCH /cart/update/{id}   → Modifier la quantité
DELETE /cart/remove/{id}  → Supprimer un article
```

4.4 Paiement (Front/CheckoutController)

Le paiement est délégué à **Stripe Checkout**. Le contrôleur crée une session Stripe et redirige l'utilisateur vers la page de paiement hébergée par Stripe.

- Création de la commande en statut `pending`
- Redirection vers Stripe Checkout
- Retour sur `/checkout/success` ou `/checkout/cancel`
- Confirmation réelle effectuée par webhook (voir §4.7)

4.5 Espace Entreprise (Company/*)

L'espace entreprise est protégé par le middleware `EnsureUserIsCompany` et `EnsureCompanyIsSubscribed`.

Fonctionnalité	Contrôleur
Tableau de bord (commandes, stats)	DashboardController
CRUD annonces	Company/ListingController
Marquer une ligne de commande préparée	Company/OrderController
Gérer les retraits (code OTP)	Company/PickupController
Onboarding Stripe Connect	Company/StripeConnectController

4.6 Demande d'upgrade (UpgradeRequestController)

Un utilisateur client peut demander à devenir entreprise via un formulaire de demande :

1. L'utilisateur renseigne son SIRET
2. Le système appelle `SireneApiService` pour vérifier le SIRET auprès de l'INSEE
3. Les données de l'entreprise sont pré-remplies (nom, adresse, activité)
4. L'utilisateur téléverse des documents justificatifs
5. L'administrateur examine et valide/rejette la demande depuis l'application lourde

4.7 Webhooks Stripe (Webhook/StripeWebhookController)

Les webhooks Stripe permettent une confirmation asynchrone des paiements. La route est exemptée de CSRF :

```
POST /stripe/webhook
```

Événements traités : `checkout.session.completed` → passage de la commande en statut `paid`. Il y a un `StripeWebhookController` utilisé pour les webhooks Stripe dans le dossier `Webhook/` (pour `web.php`) et à la racine de `Controllers/` (pour `api.php`).

4.8 Service de code de retrait (PickupCodeService)

Ce service génère et valide des codes OTP à 8 chiffres pour les retraits en magasin :

- Génération d'un code aléatoire sécurisé via `random_int()`
- Stockage d'un hash HMAC-SHA256 (jamais le code en clair)
- Validité de 7 jours, révocation automatique lors de la réémission
- Vérification atomique via transaction DB avec `lockForUpdate()`

```
// Extrait simplifié de PickupCodeService.php
$hash = hash_hmac('sha256', "{$orderId}:{$code}:{$nonce}", config('app.key'));
// Vérification : utilisation de hash_equals() (timing-safe)
```

4.9 Service SIRENE (SireneApiService)

Ce service interroge l'API SIRENE V3.11 de l'INSEE pour vérifier l'existence d'un établissement par son SIRET :

- Support de 4 modes d'authentification (Bearer, X-INSEE-Api-Key-Integration, Authorization, X-INSEE-Api-Key)
- Support OAuth2 Client Credentials en fallback
- Extraction automatique du nom, adresse et activité de l'entreprise
- Gestion des erreurs métier (SIRET invalide, non trouvé, API indisponible)

4.10 Espace Administrateur (Admin/*)

Accessible uniquement a l'utilisateur de L'application lourde.

4.11 API Application Lourde (DesktopAdminController)

Le `DesktopAdminController` gère toute l'API REST sécurisée destinée à l'application de bureau (lourde) développée en Avalonia. Ce module expose les différents endpoints nécessaires pour l'administration distante à travers un canal sécurisé.

5. Sécurité

5.1 Contrôle d'accès

Middleware	Protection
`auth`	Vérification de l'authentification
`verified`	Vérification de l'e-mail avant accès
`EnsureUserIsCompany`	Restriction aux utilisateurs rôle entreprise
`EnsureUserIsAdmin`	Restriction aux administrateurs
`EnsureCompanyIsSubscribed`	Abonnement Stripe actif requis pour gestion des annonces
`EnsureDesktopAdminToken`	Protection de l'API REST de l'app lourde via le token `desktop.admin.token`

5.2 Protection des formulaires

- **CSRF** : protection automatique Laravel sur toutes les routes POST/PATCH/DELETE (sauf webhook Stripe explicitement exclus)
- **Form Requests** : validation centralisée dans `app/Http/Requests/`
- **Mass Assignment** : chaque modèle définit une liste `$fillable` explicite

5.3 Sécurité des codes OTP

- Codes jamais stockés en clair (stockage HMAC-SHA256)
- Comparaison avec `hash_equals()` (résistant aux attaques temporelles)
- Nonce unique par émission → chaque code est non-réutilisable
- Transaction DB avec verrou (`lockForUpdate`) → protège contre les conditions de course
- Expiration automatique après 7 jours

5.4 Sécurité des paiements

- Les paiements ne sont jamais confirmés côté client : seul le webhook Stripe côté serveur valide une commande
- Vérification de la signature Stripe sur les webhooks (`Stripe\WebhookSignature::verifyHeader`)
- Les données de carte ne transitent jamais par l'application (hébergées par Stripe)

5.5 Autres mesures

- Mots de passe hashés avec Bcrypt (cast Laravel `'password' => 'hashed'`)
- Variables sensibles stockées dans `.env` (jamais dans le code)
- Fichiers uploads stockés hors `public/` , accès via Storage::url()
- Analyse statique PHPStan niveau strict (aucune variable non typée tolérée)

6. Intégration Stripe

6.1 Stripe Checkout (Paiement client)

Le flux de paiement utilise Stripe Checkout en mode *hosted page* :

```
Client → Panier → /checkout → [Création session Stripe] → Stripe Hosted Page
            ↓
            Paiement réussi / annulé
            ↓
            Webhook POST /stripe/webhook → MAJ commande
```

6.2 Stripe Connect (Paiement vendeur)

Chaque entreprise possède un compte Stripe Connect qui lui permet de recevoir les paiements directement. L'onboarding est guidé :

1. L'entreprise accède à `/company/connect/onboarding`
2. Le système crée un compte Connect Stripe et génère un lien d'onboarding
3. L'entreprise complète son profil Stripe (identité, banque)
4. Retour sur `/company/connect/return` → activation des paiements

6.3 Abonnements Stripe

Pour accéder à la gestion des annonces, l'entreprise doit souscrire un abonnement mensuel via Stripe Subscriptions :

- Route : `GET /subscription/subscribe`
- Succès : `GET /subscription/success`
- L'abonnement est enregistré dans `company_profiles` (stripe_subscription_id, stripe_subscription_status)
- Vérification d'expiration via `subscription_ends_at`

7. Tableau des routes principales

Méthode	URI	Contrôleur	Authentification
GET	/	Front/ListingController@index	Non
GET	/boutique	Front/ListingController@catalog	Non
GET	/listings/{id}	Front/ListingController@show	Non
GET/POST	/contact	Front/ContactController	Non
*	/cart/*	Front/CartController	Non
GET	/checkout	Front/CheckoutController@checkout	auth
GET/POST	/checkout/shipping	Front/CheckoutController	auth
GET	/checkout/success	Front/CheckoutController@success	auth
GET	/subscription/*	Front/SubscriptionController	auth
GET	/orders	Front/OrderController@index	auth
GET	/upgrade-request	UpgradeRequestController@create	auth
GET	/upgrade-request/verify-siret	UpgradeRequestController@verifySiret	auth
GET	/company/dashboard	DashboardController@index	auth + entreprise
CRUD	/company/listings	Company/ListingController	auth + entreprise + abonné
POST	/company/pickup/verify	Company/PickupController@verify	auth + entreprise
POST	/company/pickup/orders/{order}/mark-ready	Company/PickupController@markReady	auth + entreprise
POST	/company/pickup/orders/{order}/resend-code	Company/PickupController@resendCode	auth + entreprise
GET	/company/communes/search	Company/CommuneController@search	auth + entreprise
GET	/company/connect/onboarding	Company/StripeConnectController	auth + entreprise
PATCH	/profile/company-fulfillment	ProfileController	auth + entreprise
GET	/admin/requests	Admin/UpgradeRequestController@index	auth + admin
GET	/admin/users	Admin/UserController@index	auth + admin
API	/api/desktop-admin/*	DesktopAdminController	desktop.admin.token
POST	/stripe/webhook	Webhook/StripeWebhookController	Signature Stripe

8. Guide d'installation

> Pré-requis : PHP 8.2+, Composer, Node.js 18+, un compte Stripe (clés API).

8.1 Installation rapide (via script composer)

```
git clone <repo> site-test-marchand
cd site-test-marchand
composer run setup
```

Le script `setup` exécute automatiquement :

1. `composer install` – installation des dépendances PHP
2. Copie de `.env.example` vers `.env`
3. `php artisan key:generate` – génération de la clé d'application
4. `php artisan migrate` – création du schéma BDD
5. `npm install && npm run build` – assets frontend

8.2 Configuration des variables d'environnement

```
APP_KEY=base64:...          # Généré automatiquement
STRIPE_KEY=pk_test_...      # Clé publique Stripe
STRIPE_SECRET=sk_test_...   # Clé secrète Stripe
STRIPE_WEBHOOK_SECRET=whsec_...  # Secret webhook Stripe
STRIPE_MONTHLY_PRICE_ID=price_... # ID Prix Stripe Mensuel
STRIPE_YEARLY_PRICE_ID=price_...  # ID Prix Stripe Annuel
SIRENE_CLIENT_ID=...        # ID Client API INSEE
SIRENE_CLIENT_SECRET=...    # Secret Client API INSEE
SIRENE_BASE_URL=...         # Base URL API INSEE
SIRENE_TOKEN_URL=...        # Token URL API INSEE
```

8.3 Lancement en développement

```
composer run dev
```

Lance en parallèle : serveur PHP, queue listener, et Vite (hot reload).

8.4 Tests

```
composer run test
# ou directement
php artisan test
```

9. Pistes d'amélioration

Piste	Priorité	Description
Livraison à domicile	**Haute**	Intégration d'une API transporteur (La Poste, Colissimo)

Piste	Priorité	Description
Système d'avis	Moyenne	Permettre aux clients de noter les vendeurs et les produits
Notifications temps réel	Moyenne	WebSockets (Laravel Reverb) pour les mises à jour de commandes
Recherche avancée	Moyenne	Intégration Algolia ou Laravel Scout pour la recherche full-text
Application mobile	Basse	API REST + application mobile (Flutter/React Native)
Tests E2E	Basse	Tests de bout en bout avec Laravel Dusk ou Playwright

10. Glossaire

Terme	Définition
MVC	Model-View-Controller : patron architectural séparant les données, l'interface et la logique
ORM	Object-Relational Mapper : Eloquent dans Laravel, permet de manipuler la BDD via des objets PHP
Marketplace	Plateforme permettant à plusieurs vendeurs de proposer leurs produits à des acheteurs
Stripe Connect	Fonctionnalité Stripe permettant à des sous-comptes (vendeurs) de recevoir des paiements
Stripe Checkout	Page de paiement hébergée par Stripe (PCI-DSS compliant)
Webhook	Notification push envoyée par Stripe vers l'application lors d'un événement (paiement confirmé)
OTP	One-Time Password : code à usage unique (ici : code de retrait en magasin)
HMAC	Hash-based Message Authentication Code : empreinte cryptographique utilisée pour sécuriser les codes OTP
SIRET	Numéro d'identification d'un établissement français (14 chiffres)
INSEE SIRENE	Base de données officielle des entreprises françaises, accessible via API
Migration	Fichier PHP décrivant une évolution incrémentale du schéma de base de données
Middleware	Composant Laravel interceptant les requêtes HTTP (authentification, autorisation...)
Seeder	Script Laravel permettant de peupler la base de données avec des données de test
PSR-12	Standard de codage PHP enforced par Laravel Pint